Week 1 Workshop

Introduction to the Shell

Objectives

- To explain shell commands using context: user, machine, current directory, shell, environment, and visible files.
- To reason about paths, filesystem state, filename patterns, and permissions using concrete shell examples.
- To distinguish programs, processes, shell builtins, external commands, explicit paths, and `$PATH` lookup.
- To use streams, redirection, pipes, exit status, and error messages to diagnose command-line behaviour.

For all activities, work in groups of 3 – 4.

How to use this handout. Each activity helps you:

1. record what changed or what was observed;
2. explain the observation using the concept from the slides;
3. apply the same idea to a nearby case.

Before you arrive. Complete **Task 0 – Set Up Your Machine** in the Week 1 pre-reading. You should already have a working Linux shell, and the Week 1 starter unzipped at `~/fit2109/week1-rescue`, before this workshop begins.

New commands you will use today. Some commands in today's activities were not covered in the Week 1 pre-reading or seminar slides. Use this reference before you start so none of them is a surprise. For each command, an *option* (or *flag*, written with a leading `-`) changes how the command behaves, and an *argument* is the thing it acts on.

Your machine and shell

- `whoami` — prints your current username. Takes no options or arguments; just run `whoami`.
- `hostname` — prints the name of the computer you are working on. Run it on its own.

Processes

- `echo $$` — `echo` prints whatever follows it, and `$$` is a shell variable holding the process ID (PID) of the shell you are typing into. So `echo $$` prints that PID.
- `ps` — lists running processes. Two options are useful today:
  - `-p PID` selects a single process by its PID. Writing `-p $$` means "the current shell".

  - `-o fields` chooses which columns to show, given as a *comma-separated list* of field names: `pid` (process ID), `ppid` (parent process ID), and `comm` (command name).

  You can ask for one field (`-o pid`) or several together (`-o pid,ppid,comm`). So `ps -p $$ -o pid,ppid,comm` shows the PID, parent PID, and name of just your own shell. Here `pid,ppid,comm` are the field names given to `-o`; they are *not* separate commands.

- `head` — prints only the first lines of its input (10 by default). Add `-n` and a number to change how many, e.g. `head -n 5`. It is handy at the end of a pipe to preview long output, as in `ps -ax | head`.

Searching and summarising text (the Activity 4 pipeline)

- `grep pattern file` — prints only the lines that contain `pattern`. For example `grep "ERROR" app.log` prints the error lines. In a pipeline it reads from the command before it: `... | grep "ERROR"`.
- `sort` — sorts lines into order, alphabetically by default. Options: `-n` sorts numerically (by number value, not as text) and `-r` reverses the order. They combine, so `sort -nr` sorts numerically from largest to smallest.
- `uniq -c` — collapses *adjacent* identical lines into one, and `-c` adds a count in front of each. Because it only compares neighbouring lines, sort the input first: `sort | uniq -c`.
- `wc -l` — `wc` counts words, lines, and characters; the `-l` option selects just the line count. It is usually the last stage of a pipeline: `... | wc -l`.

Redirection and exit status

- `>` and `2>` — `>` sends a command's normal output (standard output) to a file, while `2>` sends its error messages (standard error) to a file. For example `ls missing.txt > out.txt 2> err.txt` puts normal output in `out.txt` and errors in `err.txt`.
- `$?` — a special variable holding the exit status of the command you just ran. `0` means success; any other number means failure. The next command overwrites it, so read it immediately with `echo $?`.

Activity 1: Shell Context and Location

Focus. Use this activity to connect shell context, filesystem location, and path vocabulary.

A. Decode the shell context. For each command, write what information it reveals and why that information might matter when debugging.

Command	What it tells you	Why it matters
`whoami`		
`hostname`		
`pwd`		
`echo "$SHELL"`		
`ls`		

B. Command anatomy. For each command below, identify the command name, any option, and any argument.

```
pwd
ls
ls -l scripts
cd data/raw
mkdir -p week1/data/raw
```

C. Predict the destination. Suppose your current directory is:

```
/Users/student/fit2109/week1/data/raw
```

For each command, predict the new working directory, or explain why the command might fail.

Command	New working directory	Reason
`cd ..`		
`cd ../..`		
`cd ~/fit2109/week1`		
`cd ./data/raw`		

D. Explain to another group. In two sentences, explain why many command-line mistakes are really location mistakes. Use `pwd`, relative paths, and absolute paths in your explanation.

Activity 2: Filesystem State, Patterns, and Permissions

Focus. Use this activity to reconstruct filesystem state changes, filename-pattern expansion, and permission changes.

A. Trace the filesystem. Assume you start in an empty directory. Fill in what changes after each command.

1. `touch notes.txt`
   What changed? _____
2. `mkdir scripts backups`
   What changed? _____
3. `cp notes.txt backups/notes-copy.txt`
   What changed? _____
4. `mv notes.txt summary.txt`
   What changed? _____

5. `cp summary.txt scripts/summary-backup.txt`
   What changed? _____
6. `rm backups/notes-copy.txt`
   What changed? _____
7. `ls -R`
   What does this inspect? _____

B. Draw the final tree. Draw the final directory tree after Part A. Your tree should show which files are in the current directory, `scripts/`, and `backups/`.

C. Pattern expansion. Suppose the current directory contains:

```
summary.txt  notes.txt  run.sh  clean.sh  scripts  backups
```

Before the command runs, what does the shell expand each pattern into?

Command	Expanded filenames	No-match risk?
`ls *.txt`		
`ls *.sh`		
`cp *.txt backups/`		
`ls *.csv`		

Bash and Zsh can behave differently when a pattern has no match. We will study expansion and quoting properly in Week 2.

D. Permission diagnosis. A file named `scripts/run.sh` has this content:

```
echo Hello FIT2109
```

Now examine the two permission strings:

```
-rw-r--r--  1 student staff 19 Mar  3 10:20 scripts/run.sh
-rwxr-xr--  1 student staff 19 Mar  3 10:22 scripts/run.sh
```

1. What changed between the two permission strings?
2. Why did `./scripts/run.sh` fail before the permission change?
3. What exactly does `chmod u+x scripts/run.sh` add?
4. Does `chmod u+x` change the script contents? Explain.
5. In `-rwxr-xr--`, can "others" run the file? Which three characters tell you?

Activity 3: Execution, Processes, and `$PATH`

Focus. Use this activity to connect processes, shell builtins, external commands, `$PATH`, and explicit paths.

A. Program or process? For each item, decide whether it is best described as a stored program, a running process, shell context, or command lookup information.

Item	Category	Reason
`scripts/run.sh` on disk		
The shell running in your terminal		
The number printed by `echo $$`		
The row printed for the current shell by `ps`		
The list printed by `echo "$PATH"`		

B. Read process output. Suppose `ps -p $$ -o pid,ppid,comm` prints:

```
  PID  PPID COMM
41820 41798 zsh
```

1. What is the PID of the current shell?
2. What does PPID refer to?
3. What does `comm` tell you?
4. Why can `ps -ax | head` show processes that are not your shell?

C. Command lookup. Assume you are in the project root and `scripts/run.sh` exists and is executable.

```
$ run.sh
zsh: command not found: run.sh

$ ./scripts/run.sh
Hello FIT2109
```

1. Why does `run.sh` fail even though the file exists?
2. Why does `./scripts/run.sh` work?
3. What role does `$PATH` play when you type a command name?
4. Why is the current directory usually not searched automatically?
5. Why is `cd` different from `ls` when you inspect them with `type`?

D. Bash and Zsh note. Week 1 commands such as `pwd`, `ls`, `cd`, redirection, and pipes behave the same for our purposes in Bash and Zsh. Scripts in this unit will use Bash with `#!/usr/bin/env bash`. Why is that useful even if your interactive shell is Zsh?

Activity 4: Streams, Pipelines, Exit Status, and Failure Clues

Focus. Use this activity to connect streams, redirection, pipelines, exit status, and error messages.

A. Where did the stream go? For each command, identify standard input, standard output, and standard error where relevant.

1. `echo "first line" > notes.txt`
   stdin: _____ stdout: _____ stderr: _____
2. `echo "second line" >> notes.txt`
   stdin: _____ stdout: _____ stderr: _____
3. `sort < notes.txt`
   stdin: _____ stdout: _____ stderr: _____
4. `ls missing.txt > output.txt 2> errors.txt`
   stdin: _____ stdout: _____ stderr: _____

B. Build a pipeline from a question. Suppose `app.log` contains:

```
INFO Login ok
WARN Disk space low
ERROR Database unavailable
WARN Disk space low
ERROR Timeout contacting API
ERROR Database unavailable
```

1. Write a command that prints only the `ERROR` lines.
2. Write a command that counts the `ERROR` lines.
3. Write a command that counts each distinct `ERROR` line.
4. Write a command that saves the distinct error summary to `error-summary.txt`.

C. Explain the pipeline. For your answer to Part B.3, explain what each stage contributes.

Pipeline stage	What this stage does
`grep`	
`sort`	
`uniq -c`	
`sort -nr`	

D. Exit status and failure clues. For each command pair, predict the exit status and explain the clue.

1. `ls app.log`
   Expected `$?`: _____ Why? _____
2. `ls missing.log`
   Expected `$?`: _____ Why? _____

3. `grep "ERROR" missing.log | wc -l`
   Expected `$?`: _____ Why? _____
4. `grep "CRITICAL" app.log`
   Expected `$?`: _____ Why? _____

In many shells, `$?` after a pipeline reports the status of the last command. In Bash scripts, `set -o pipefail` makes the pipeline fail if any stage fails.

E. Turn an error into a next step. Choose one failing command from this activity. Write:

1. the exact error message;
2. what object the error message points to;
3. one command you would run next, such as `pwd`, `ls`, `man`, or `--help`;
4. one precise AI question that includes the command, the error message, what you expected, and your current directory.

Final Checkpoint Preparation

Before the final polls, make sure your group can answer:

1. What command moves up one directory?
2. In `-rwxr-xr--`, can others run the file?
3. Why might `run.sh` fail while `./scripts/run.sh` works?
4. What does exit status `0` usually mean?
5. What is one shell idea you want to practise more this week?